

Data Modeling Lecture Notes

What and Why of Data Modeling?

What is Data Modeling?

- Data modeling is the process of creating a visual representation of data and its relationships within an information system.
- It's a blueprint for how data will be stored and accessed.
- It involves identifying data objects and their relationships.
- It helps in understanding business requirements and translating them into a database structure.
- It is used to represent the data entities and the relation between the entities.

Why is Data Modeling Important?

- **Improves Data Quality:** Reduces redundancy and inconsistency.
- **Enhances Data Consistency:** Ensures that data is accurate and reliable.
- **Facilitates Communication:** Provides a common language for developers, business users, and stakeholders.
- **Supports Decision Making:** Makes it easier to retrieve and analyze data.
- **Simplifies Database Design:** Provides a clear roadmap for database development.
- **Reduces Development Costs:** Prevents errors and rework.
- **Increases Scalability:** Designs that can adapt to changing needs.
- **Improves Performance:** Optimizes data storage and retrieval.

Relational vs. NoSQL Databases

Relational Databases

- Based on the relational model, data is organized into tables with rows and columns.
- Use SQL (Structured Query Language) for defining and manipulating data.
- Examples: MySQL, PostgreSQL, Oracle, Microsoft SQL Server.
- Follow ACID properties (Atomicity, Consistency, Isolation, Durability).
- Well-suited for structured data and complex queries.
- Schema is predefined.

NoSQL Databases

- "Not Only SQL" databases.
- Handle unstructured or semi-structured data.
- Various types:

- **Key-Value:** Data is stored as key-value pairs (e.g., Redis, Memcached).
- **Document:** Data is stored in JSON-like documents (e.g., MongoDB, CouchDB).
- **Column-Family:** Data is stored in column families (e.g., Cassandra, HBase).
- **Graph:** Data is stored as nodes and edges (e.g., Neo4j).
- Offer flexibility, scalability, and high performance.
- Follow BASE properties (Basically Available, Soft state, Eventual consistency).
- Schema is dynamic.

Relational vs. NoSQL: Key Differences

Feature	Relational Databases	NoSQL Databases
Data Model	Tables with rows and columns	Key-value, document, column, graph
Query Language	SQL	Varies (often not SQL)
Consistency	ACID	BASE
Schema	Fixed, predefined	Dynamic, flexible
Scalability	Vertical	Horizontal
Best For	Structured data, complex queries	Unstructured data, high performance

Concept of Entity, Entity Type, and Attributes

Entity

- A thing or object in the real world that is distinguishable from other objects.
- Examples: A student, a book, a customer, an order.

Entity Type

- A collection of entities that share common properties or characteristics.
- Also known as an entity set.
- Represented as a rectangle in an Entity-Relationship (ER) diagram.
- Example: The entity type "Student" represents all students in a school.

Attributes

- Properties or characteristics of an entity type.
- Describe the entity.

- Represented as ovals in an ER diagram.
- Example: For the entity type "Student," attributes might include:
 - StudentID
 - Name
 - Age
 - Address
 - Major

Types of Attributes

- **Simple (Atomic):** Cannot be further divided (e.g., Age, Name).
- **Composite:** Can be divided into sub-attributes (e.g., Address can be divided into Street, City, State, Zip).
- **Single-valued:** Has only one value for an entity (e.g., Age).
- **Multi-valued:** Can have multiple values for an entity (e.g., Phone Numbers).
- **Derived:** Calculated from other attributes (e.g., Age can be derived from Date of Birth).
- **Key Attribute:** Uniquely identifies an entity within an entity set.
 - Can be a *Super Key*, *Candidate Key* or *Primary Key*

Database Keys

What are Keys?

- In a database, a key is an attribute or a set of attributes that helps to identify a record (row) in a table.
- Keys are crucial for establishing relationships between tables and ensuring data integrity.

Types of Keys

- **Super Key:**
 - Any set of attributes that uniquely identifies each row in a table.
 - A super key may contain redundant attributes.
 - Example: In a "Student" table, {StudentID}, {StudentID, Name}, and {Email, StudentID} are all super keys.
- **Candidate Key:**
 - A minimal super key, meaning it is a super key with no redundant attributes.
 - A candidate key uniquely identifies each row in a table, and no subset of its attributes can do so.
 - A table can have multiple candidate keys.
 - Example: In a "Student" table, if both StudentID and Email uniquely identify each student, then both StudentID and Email are candidate keys.

- **Primary Key:**
 - A candidate key that is chosen as the principal identifier for the table.
 - It is used to uniquely identify each row in the table and is used to reference the table from other tables.
 - A table can have only one primary key.
 - Example: In a "Student" table, if both StudentID and Email are candidate keys, we might choose StudentID as the primary key.

Composite Key

- A key that consists of two or more attributes that together uniquely identify a row in a table.
- It is used when a single attribute is not sufficient to guarantee uniqueness.
- A composite key can be a candidate key or a primary key.
- Example:
 - Table: **OrderDetails**
 - Attributes: **OrderID, ProductID, Quantity**
 - Here, the combination of **OrderID** and **ProductID** together can uniquely identify a row in the **OrderDetails** table. So, {OrderID, ProductID} is a composite key.

Composite Primary Key

- A primary key that consists of two or more attributes.
- It uniquely identifies each row in a table, and no subset of the attributes can do so.
- Example:
 - Table: **Enrollments**
 - Attributes: **StudentID, CourseID, EnrollmentDate**
 - Here, the combination of **StudentID** and **CourseID** uniquely identifies each enrollment record.
 - Therefore, {StudentID, CourseID} is a composite primary key.

Foreign Key

- An attribute or a set of attributes in one table that refers to the primary key in another table.
- It establishes a link between two tables and enforces referential integrity.
- The table containing the foreign key is called the "referencing" or "child" table, and the table containing the primary key is called the "referenced" or "parent"

table.

- Example:
 - Table: **Orders**
 - Attributes: **OrderID, CustomerID, OrderDate**
 - Table: **Customers**
 - Attributes: **CustomerID, Name, Address**
 - Here, **CustomerID** in the **Orders** table is a foreign key that references the **CustomerID** (primary key) in the **Customers** table. This establishes a relationship between orders and customers.

●

Candidate Key vs. Primary Key

Feature	Candidate Key	Primary Key
Definition	A minimal super key that uniquely identifies each row in a table.	A candidate key chosen as the principal identifier for a table.
Uniqueness	Uniquely identifies rows.	Uniquely identifies rows.
Number	A table can have multiple candidate keys.	A table can have only one primary key.
Null Values	Can contain NULL values (unless explicitly constrained).	Cannot contain NULL values.
Purpose	Represents a potential primary key.	Uniquely identifies rows and is used for relationships with other tables.
Selection	A table can have one or more candidate keys.	Chosen from the candidate keys.

Database Normalization

Functional Dependency

- A fundamental concept in database design.
- Describes the relationship between attributes in a relation (table).
- Formally, a functional dependency exists between two sets of attributes, X and Y, within a relation R, if and only if each value of X is associated with exactly one value of Y.
- If a functional dependency exists, we say "Y is functionally dependent on X".

- Denoted as $X \rightarrow Y$, where X is the determinant and Y is the dependent.
- Example: In a "Student" table, if StudentID determines Name, we write $\text{StudentID} \rightarrow \text{Name}$.

Types of Functional Dependencies

- **Trivial Functional Dependency:**

- Occurs when an attribute determines itself or a subset of itself.
- If Y is a subset of X , then the functional dependency $X \rightarrow Y$ holds.
- Example:
 - $\{\text{StudentID}, \text{Name}\} \rightarrow \text{StudentID}$ (Since StudentID is a subset of $\{\text{StudentID}, \text{Name}\}$)
 - $\text{Name} \rightarrow \text{Name}$

- **Non-Trivial Functional Dependency:**

- Occurs when an attribute determines another attribute or set of attributes that are not subsets of itself.
- If $X \rightarrow Y$, and Y is not a subset of X , then the functional dependency $X \rightarrow Y$ is non-trivial.
- Example:
 - $\text{StudentID} \rightarrow \text{Name}$ (Name is not a subset of StudentID)
 - $\text{Email} \rightarrow \text{StudentID}$

- **Partial Functional Dependency:**

- Occurs when a non-key attribute is dependent on only a part of a composite primary key, not the entire key.
- Consider a relation R with attributes A , B , C , and D . If $\{A, B\}$ is the composite primary key and C is a non-key attribute, then:
 - C is partially dependent on $\{A, B\}$ if $A \rightarrow C$ or $B \rightarrow C$.
- Example:
 - Table: OrderDetails (OrderID, ProductID, ProductName, Quantity)
 - Here, $\{\text{OrderID}, \text{ProductID}\}$ is the composite primary key.
 - ProductName is only dependent on ProductID, not OrderID.
 - Thus, $\text{ProductID} \rightarrow \text{ProductName}$ is a partial functional dependency.

- **Transitive Functional Dependency:**

- Occurs when an attribute is indirectly dependent on another attribute through a third attribute.
- If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$ is a transitive dependency, provided that:
 - A is not functionally dependent on B
 - B is not functionally dependent on A
- Example:
 - $EmployeeID \rightarrow DepartmentID$
 - $DepartmentID \rightarrow DepartmentName$
 - Then, $EmployeeID \rightarrow DepartmentName$ is a transitive dependency.

What is Database Normalization?

- The process of organizing data in a database to reduce redundancy and improve data integrity.
- Involves dividing a database into two or more tables and defining relationships between them.
- Developed by Edgar F. Codd.
- Uses the concept of functional dependencies.

Goals of Normalization

- Minimize data redundancy.
- Eliminate data anomalies (insertion, update, and deletion anomalies).
- Simplify data management.
- Improve data integrity.

Normal Forms

- Rules that define how relations (tables) should be organized.
- First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF) are the most commonly used.

1NF (First Normal Form)

- A relation is in 1NF if every attribute in each tuple is atomic (single-valued).

- Eliminates repeating groups.
- Example:
 - **Unnormalized Table:**

CustomerID	Name	Phone Numbers
1	John Doe	123-456-7890, 987-654-3210

- **1NF Table:**

CustomerID	Name	Phone Number
1	John Doe	123-456-7890
1	John Doe	987-654-3210

2NF (Second Normal Form)

- A relation is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on the primary key.
- Eliminates partial dependencies.
- Requires a composite primary key.
- Example:
 - **1NF Table:**

OrderID	ProductID	ProductName	Quantity	Price
---------	-----------	-------------	----------	-------

- **2NF Tables:**

OrderID	ProductID	Quantity
---------	-----------	----------

ProductID	ProductName	Price
-----------	-------------	-------

3NF (Third Normal Form)

- A relation is in 3NF if it is in 2NF and no non-key attribute is transitively dependent on the primary key.
- Eliminates transitive dependencies.
- Example:
 - **2NF Table:**

EmployeeID	Name	DepartmentID	DepartmentName	Location
------------	------	--------------	----------------	----------

- **3NF Tables:**

EmployeeID	Name	DepartmentID
------------	------	--------------

DepartmentID	DepartmentName	Location
--------------	----------------	----------

- A relation is in 3NF if it is in 2NF and no non-key attribute is transitively dependent on the primary key.
- In simpler terms, all non-key attributes depend only on the primary key.

Denormalization Trade-offs

- **Denormalization:** The process of intentionally introducing redundancy into a database to improve performance.
- Trade-offs:
 - **Pros:**
 - Improved query performance (faster retrieval).
 - Simpler queries.
 - **Cons:**
 - Increased data redundancy.
 - Potential data inconsistency.
 - Increased storage space.
 - More complex updates.
- When to denormalize:
 - When read performance is critical.
 - For reporting and data warehousing.
 - When dealing with complex joins.